HW4: Hashing

CYB-402

Tyler Vander Mooren

1. Why do we include the message length in the calculation of the hash code?

In SHA-1 the original message length is added to the end of the padded message to keep the hash process accurate and secure. This step called Merkle–Damgård strengthening, makes sure that the hash represents both the content and its actual size. By appending a single "1" bit followed by enough zeros and then the 64-bit binary version of the original length, the final message fits perfectly into a 512-bit block. This ensures the algorithm always knows where the message truly ends and prevents certain attacks that could alter or extend the message without changing the hash.

2. SHA-1 should NOT be used for messages longer than what?

SHA-1 stores the message length in a 64-bit field, meaning it can only handle messages smaller than 2^{64} bits. That's an enormous amount of data but it's a hard limit built into the design. If a message went beyond that size the counter used to record the length would overflow and wrap around to zero, essentially breaking the padding structure and creating the potential for invalid or conflicting hashes. The 64-bit limit ensures that every valid message length can be uniquely represented and safely processed.

3. How are these numbers selected?

The initial hash values (H_0–H_4) in SHA-1 are fixed 32-bit hexadecimal constants used to start the hashing process. They are chosen to provide a balanced and nonrepetitive starting point for the internal state helping the algorithm achieve strong diffusion as it processes data. These

constants were inherited from earlier algorithms in the MD-4 and MD-5 family to maintain a similar structure and ensure consistency across implementations.

4. Why can a hash function not be used for encryption?

Hash functions are one way by design. They take input data of any length and compress it into a fixed length output with no way to reverse the process. Encryption on the other hand is meant to be reversible. It uses a secret key to transform data and another key (or the same) to recover it. Since hashing doesn't involve a key or any reversible steps, you can't decrypt a hash or recover the original message. That's why hashes are used for verifying integrity and not for keeping information secret.

5. What is meant by the strong collision resistance property of a hash function?

Strong collision resistance means it should be practically impossible to find two different inputs that generate the same hash output. In other words, even the smallest change in input should completely change the resulting output. This property is crucial for digital signatures, password storage and file integrity checks. A real example of this failing happened in 2017 with the SHAttered attack, where researchers created two different PDF files that shared the same SHA-1 hash showing that SHA-1 was no longer reliable for collision resistance (shattered.io).

6. Right or wrong: When you create a new password, only the hash code for the password is stored. The text you entered for the password is immediately discarded.

True. When you create a password the system hashes it immediately and throws away the original text version. What gets stored is a salted and strengthened hash, meaning a random value (the salt) is added to make each password and the hash is run through a slow algorithm like PBKDF2, bcrypt, scrypt, or Argon2. This combination makes it extremely difficult for attackers to reverse engineer or brute force the password even if they somehow get access to the database.

7. What is the relationship between “hash” as in “hash code” or “hashing function” and “hash” as in a “hash table”?

A hash code is the result of a cryptographic hash function it’s like a digital fingerprint used to verify integrity or authenticity. A hash table, on the other hand is a data structure that uses simple hash functions to organize and quickly find stored information. For example, a hash table might help a program quickly look up stored checksums or usernames. While a cryptographic hash code makes sure those checksums haven’t been changed or tampered with. One is about fast access while the other is about trust and integrity.

8. In 2 to 3 complete paragraphs (4 to 6 sentence paragraphs x 2), discuss when hashing is cryptographically secure? Provide at least 2 ACADEMIC references that you’ve used to support your discussion.

A cryptographically secure hash function is built so it can’t realistically be reversed or predicted even with massive computing power. To meet that standard, it must satisfy three main conditions. Preimage resistance, second-preimage resistance, and strong collision resistance. Preimage resistance means no one can figure out the original input just by looking at the hash. Second-preimage resistance means that if you already know one message and its hash, you can’t find another message that produces that same hash. Strong collision resistance takes it a step further, making it practically impossible to find any two different inputs that generate the same output. When those properties are met, a hash works like a digital fingerprint. Unique, irreversible and extremely sensitive to the point that even a single bit being changed affects it (Aumasson, 2024).

Still meeting those three conditions alone isn’t enough to guarantee long-term security. What really matters is how the algorithm’s internal design holds up against evolving

cryptanalysis. Modern hashes like SHA-2 and SHA-3 use large internal states, nonlinear mixing functions and a significant number of transformation rounds to spread out data and resist attacks like differential analysis or length extension. They're also designed with wide safety margins, extra rounds and structure that go beyond what's needed right now leaving room for future advances in computing. Over time maintaining cryptographic security comes down to strong design, open testing and the ability to adapt to new attack methods. A hash function only stays secure as long as its internal design and assumptions continue to hold up under both theory and real-world testing (Paar et al., 2024).

References

Aumasson, J.-P. (2024). *Serious Cryptography: A Practical Introduction to Modern Encryption* (2nd ed.). No Starch Press.

Paar, C., Pelzl, J., & Güneysu, T. (2024). *Understanding Cryptography: From Established Symmetric and Asymmetric Ciphers to Post-Quantum Algorithms* (2nd ed.). Springer.

Stevens, M., Bursztein, E., Karpman, P., Albertini, A., & Markov, Y. (2017). *The first collision for full SHA-1*. Google Research & CWI Amsterdam. https://shattered.io/